

Курсовой проект по курсу дискретного анализа: архиватор

Выполнил студент группы 08-303 МАИ *Арусланов Кирилл*.

Условие

Общая Необходимо реализовать два известных метода сжатия данных для сжатия одного файла. Методы сжатия выбираются из следующих групп:

- Арифметическое кодирование, кодирование по Хаффману
- LZ77, LZW, BWT+MTF+RLE

Формат запуска должен быть аналогичен формату запуска программы gzip. Должны поддерживаться следующие ключи: -c, -d, -k, -l, -r, -t, -1, -9. Должно поддерживаться указание символа дефиса в качестве стандартного ввода.

Вариант задания: LZ77 + кодирование по Хаффману

Метод решения

Программа реализует архиватор с двумя уровнями сжатия: -1 (чистый LZ77) и -9 (LZ77 + Huffman).

LZ77 работает по всему тексту, используя suffix array (SA) для оптимизации поиска совпадений. Входные данные загружаются в строку. Строится mapped-строка с терминатором для корректной сортировки суффиксов. Затем с помощью алгоритма на основе удвоения строится suffix array ($O(n \log n)$ время) и массив longest common prefix (LCP). Поиск наилучшего матча для позиции pos: сканирование соседей в SA с использованием LCP для быстрого определения длины совпадения, выбор максимальной длины с минимальным offset и порогом $\text{length} \geq 4$. Результат —

вектор токенов: либо литерал (`offset=0, length=0, ch`), либо матч (`offset > 0, length, has_char, optional ch`).

Сериализация токенов реализована через бит-пакетный формат: сначала 64 бита количества токенов, затем токены (1 бит флаг: 0 — литерал (8 бит `ch`); 1 — матч (16 бит `offset` + 16 бит `length` + 1 бит `has_char` + optional 8 бит `ch`)). После паддинга до байта получается `packed_bytes` и `valid_bits` (реальное количество бит).

Для уровня -1: в архив пишется `u64 valid_bits + packed_bytes` напрямую.

Для уровня -9: `packed_bytes` трактуется как последовательность байтов-символов (`uint32_t`), на которой строится каноническое Huffman-кодирование: подсчёт частот, построение дерева (`priority_queue`), итеративный сбор длин кодов, канонизация (сортировка по (`len, symbol`)), присвоение кодов. Записывается таблица (`u32 unique_count + unique_count × (u32 symbol + u16 len)`) + `u64 total_symbols` + сжатый битстрим.

Формат архива: заголовок 12 байт ("LZ77" + `version=1 + level=1/9 + u32 CRC32 исходных данных + reserved 2 байта`), затем для -1 — `u64 valid_bits + packed_bytes`; для -9 — `u64 valid_bits + Huffman-блок (таблица + битстрим)`.

Декомпрессия: чтение заголовка и CRC32, затем для -1 — чтение `valid_bits + packed_bytes`; для -9 — чтение `valid_bits` + декодирование Huffman в `packed_bytes`. Далее `BitReader` на `packed_bytes` с `valid_bits` восстанавливает токены (по флагу). Затем стандартная LZ77-декомпрессия: копирование по `offset/length`. В конце проверяется CRC32 распакованных данных.

Ключи командной строки обрабатываются в `main`: поддержка `-c/-d/-l/-t/-k/-r`, работа со `stdin/stdout` ("-"), рекурсивный обход директорий для `-r`. Функция `list` (-l) распаковывает архив в память для вывода статистики (уровень, количество

токенов, размеры, ratio). Функция test (-t) проверяет целостность: распаковка + сверка CRC32 без записи на диск.

Описание программы

Программа на C++ разделена на модули:

- `lz77.hpp/cpp` — структура Token, сжатие с SA/LCP, декомпрессия.
- `huffman.hpp/cpp` — каноническое Huffman-кодирование (BitWriter/Reader, encode/decode_symbols).
- `utils.hpp/cpp` — сериализация токенов, поддержка stdin/stdout.
- `archiver.hpp/cpp` — функции compress/decompress_stream, list/test_archive.
- `main.cpp` — парсинг аргументов, обработка файлов/директорий (с -r).

Дневник отладки

Изначально Huffman был обычным — перешёл на канонический для компактной таблицы. Ошибки в BitReader/Writer (паddинг, порядок битов). Отлаживал на маленьких тестах (twoletter, lorem). Проверка на stdin требовала буферизации для list/test. Сделал сериализацию более эффективной, изменив ее подход.

Тест производительности

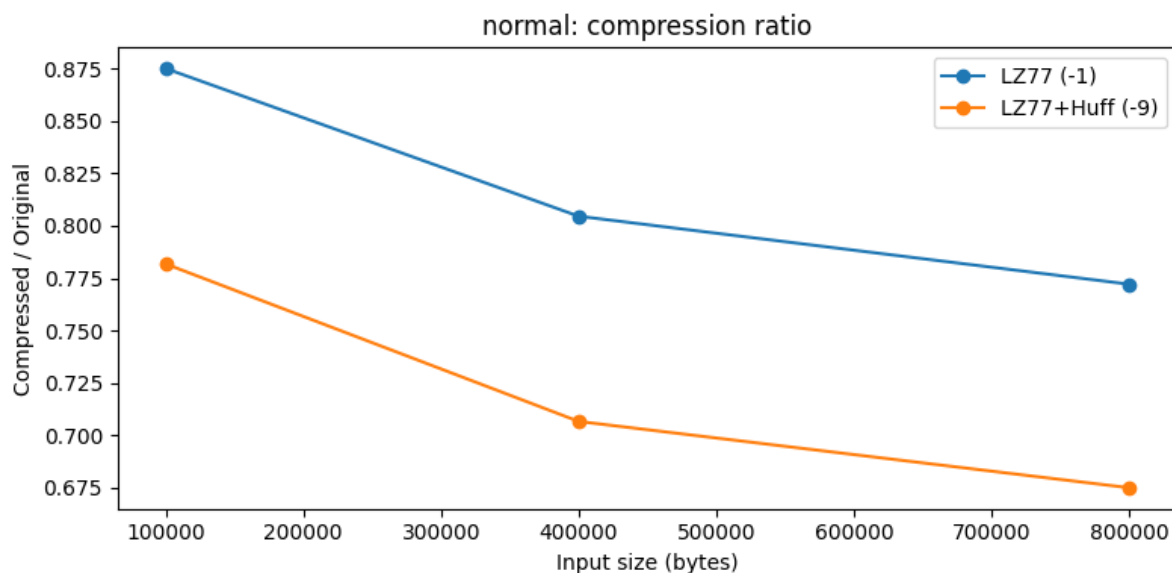
Тестирование проводилось на синтетических датасетах различного типа и реальных файлах. Измерялись время сжатия/распаковки и размер сжатого

файла. Режим -1 — чистый LZ77 с бит-пакетным форматом токенов, режим -9 — LZ77 + Huffman-кодирование бит-пакета.

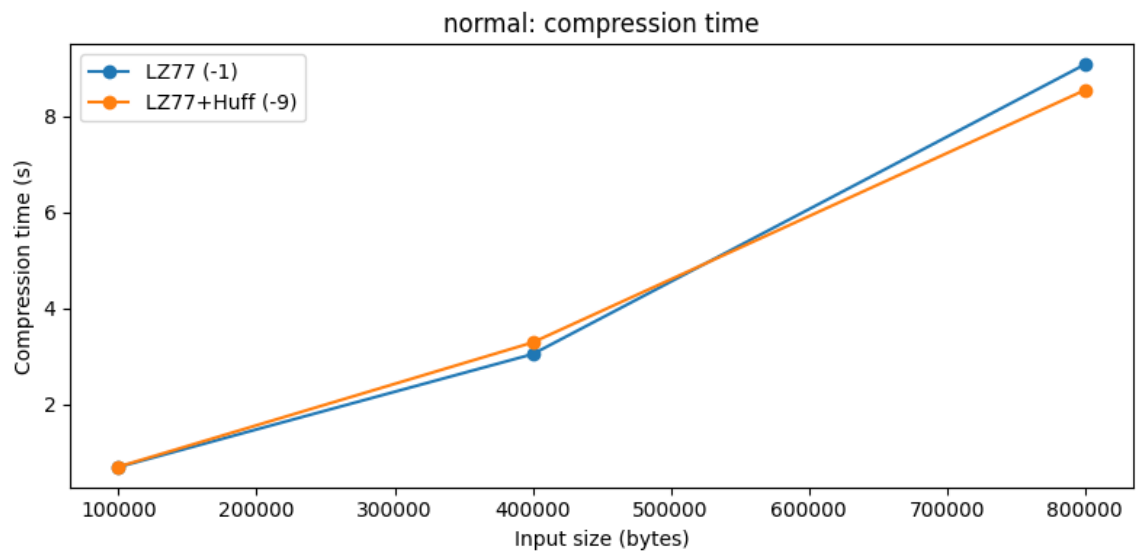
Синтетические данные: нормальное распределение

Ниже результаты теста на данных, имеющих нормальное распределение. Благодаря выраженному предпочтению к байтам около центра распределения LZ77 находит множество повторяющихся последовательностей, а Huffman эффективно сжимает частые значения.

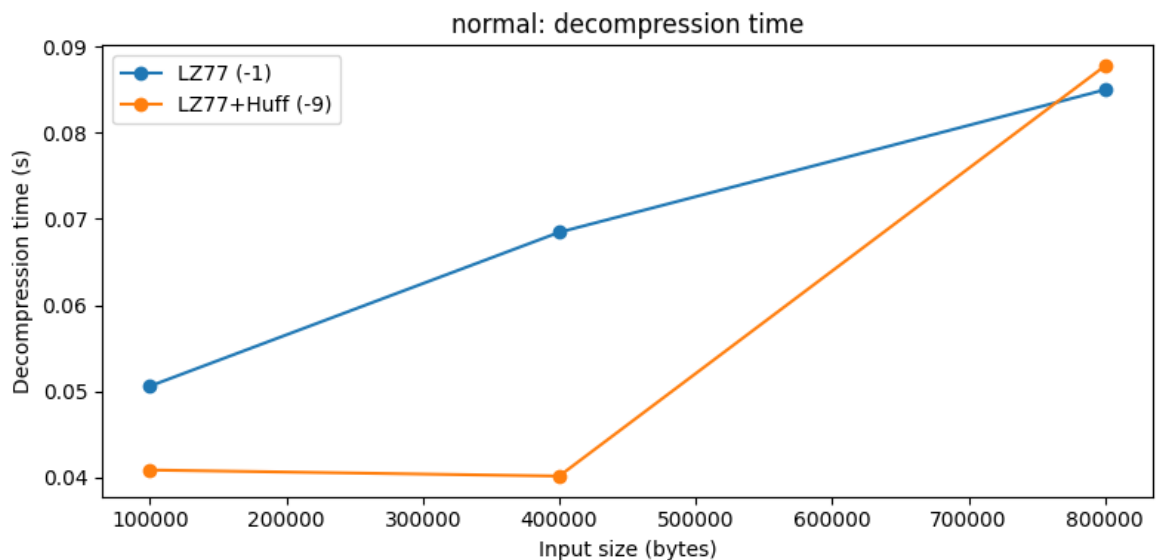
Размер	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	0.686	0.051	87 501
	-9	0.693	0.041	78 180
400 000	-1	3.041	0.068	321 828
	-9	3.284	0.040	282 681
800 000	-1	9.082	0.085	617 722
	-9	8.547	0.088	540 058



Коэффициент сжатия (normal)



Время сжатия (normal)



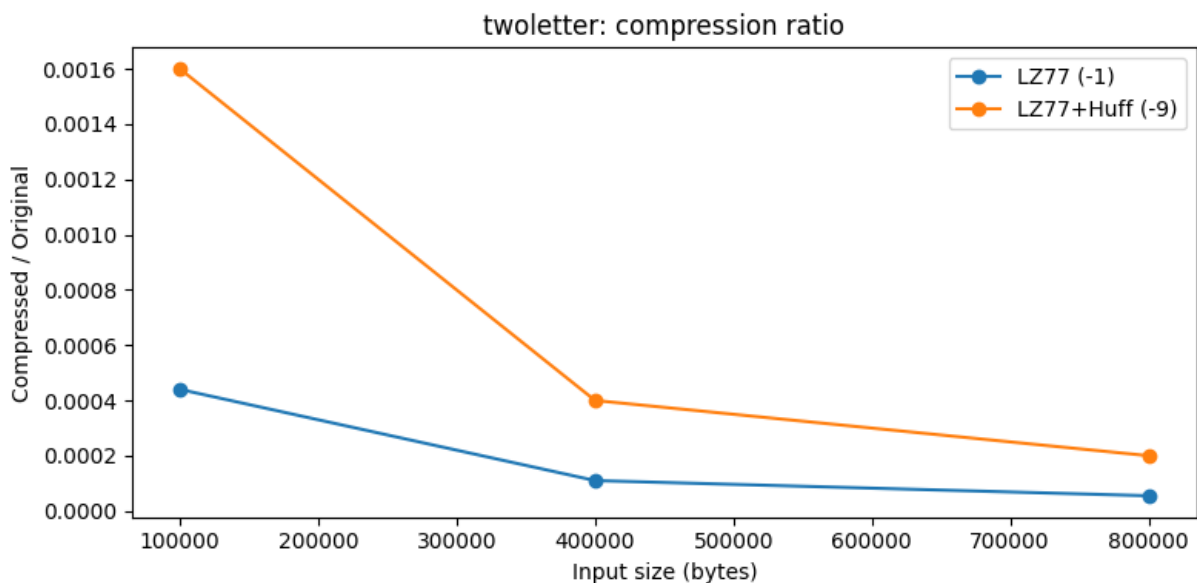
Время распаковки (normal)

Режим -9 заметно превосходит -1 (ratio ≈ 0.67 – 0.78 против 0.77 – 0.87), так как Huffman эффективно сжимает бит-пакет токенов при наличии предпочтения к определённым байтам. LZ77 также хорошо справляется благодаря кластерам похожих значений.

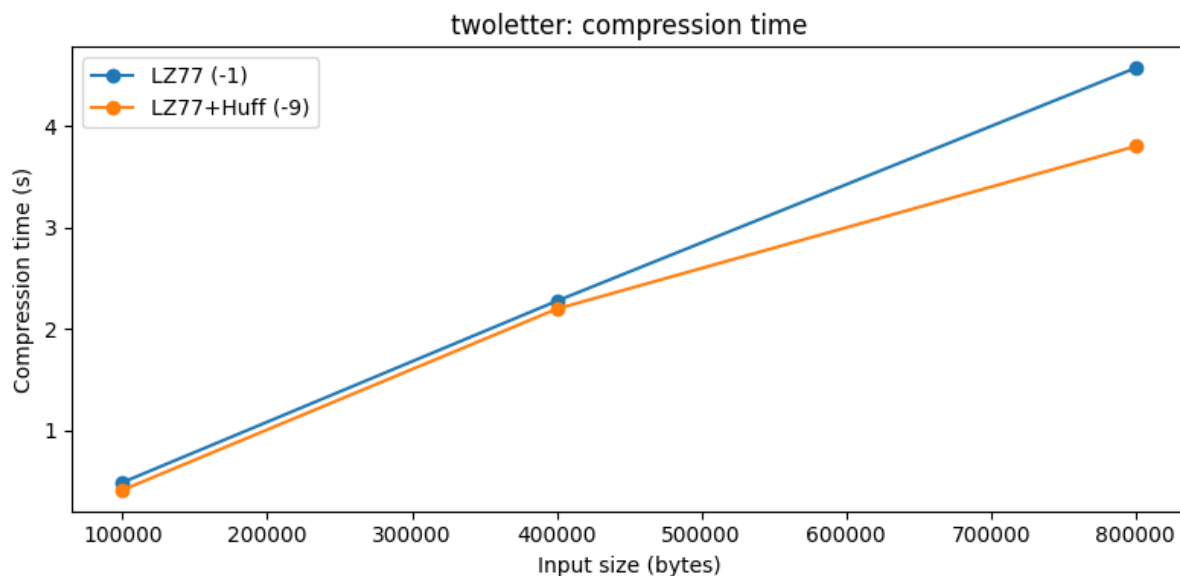
Синтетические данные: высокая повторяемость (twoletter)

Ниже результаты на данных с экстремальной повторяемостью (чередование двух байтов). Здесь ожидаемо отличное сжатие — LZ77 находит очень длинные матчи.

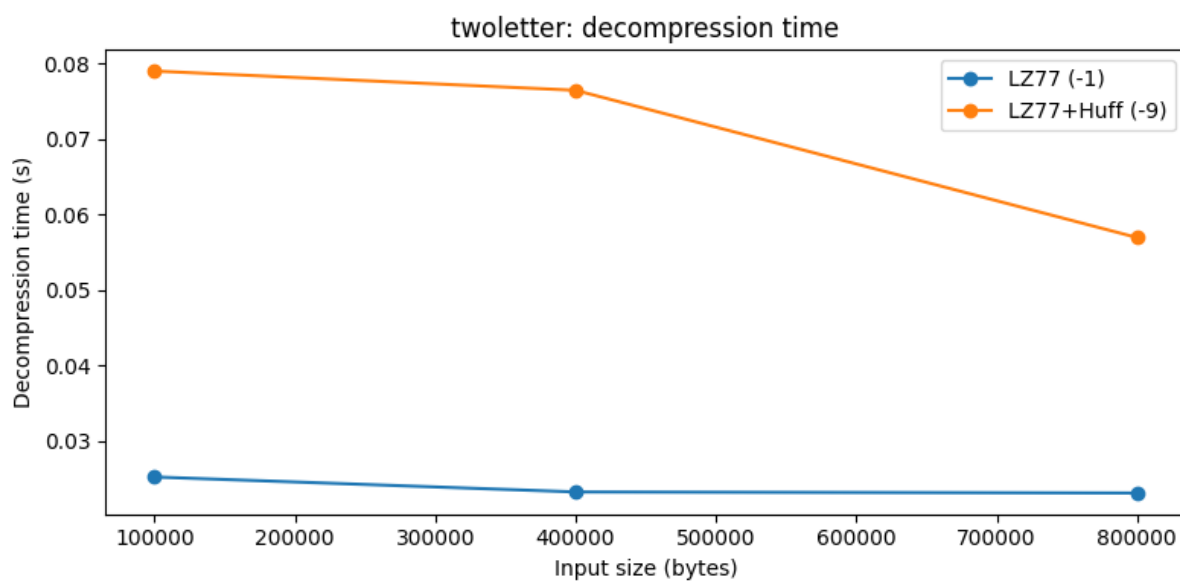
Размер	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	0.488	0.025	44
	-9	0.413	0.079	160
400 000	-1	2.274	0.023	44
	-9	2.196	0.076	160
800 000	-1	4.568	0.023	44
	-9	3.798	0.057	160



Коэффициент сжатия (twoletter)



Время сжатия (twoletter)



Время распаковки (twoletter)

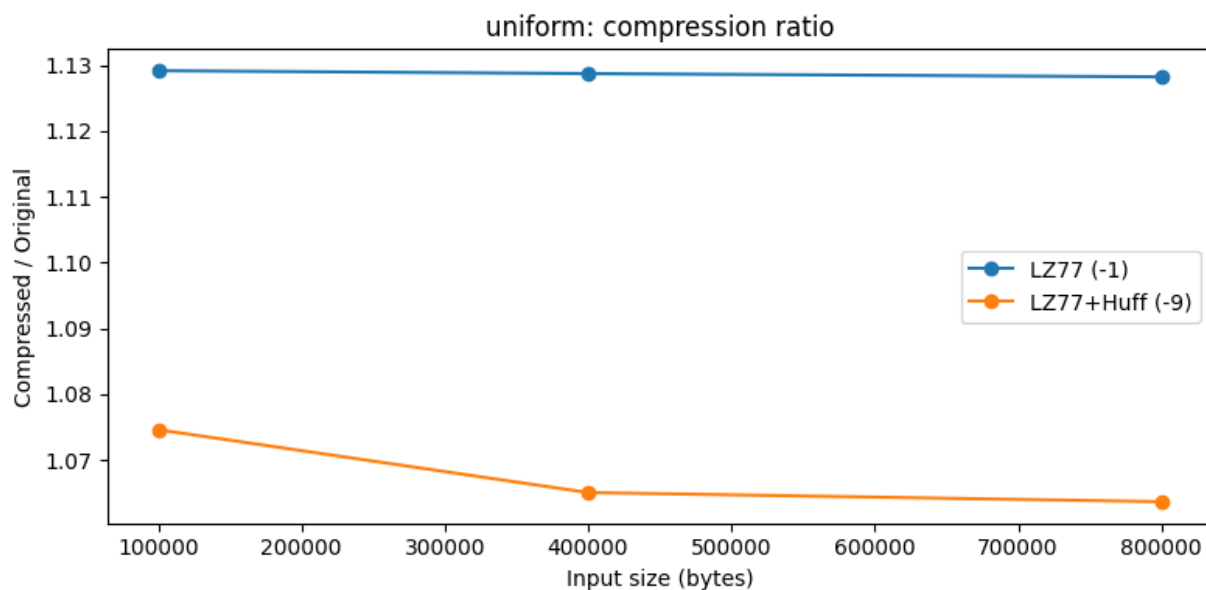
Режим -1 показывает практически идеальное сжатие (размер не зависит от исходного). Режим -9 проигрывает из-за overhead таблицы Huffman (много уникальных значений в бит-пакете токенов), хотя битстрим сжимается сильно.

Синтетические данные: равномерное распределение (uniform)

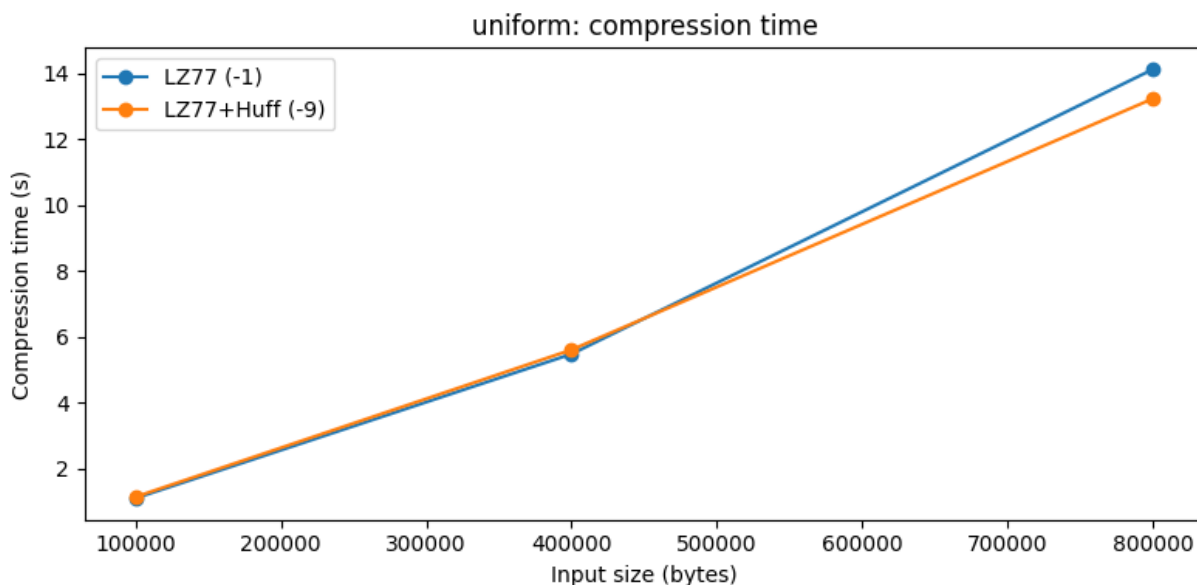
Ниже результаты на данных с равномерным распределением байтов (0–255). Здесь отсутствуют повторения и предпочтения символов —

противоположная ситуация по сравнению с нормальным распределением,
поэтому сжатие хуже.

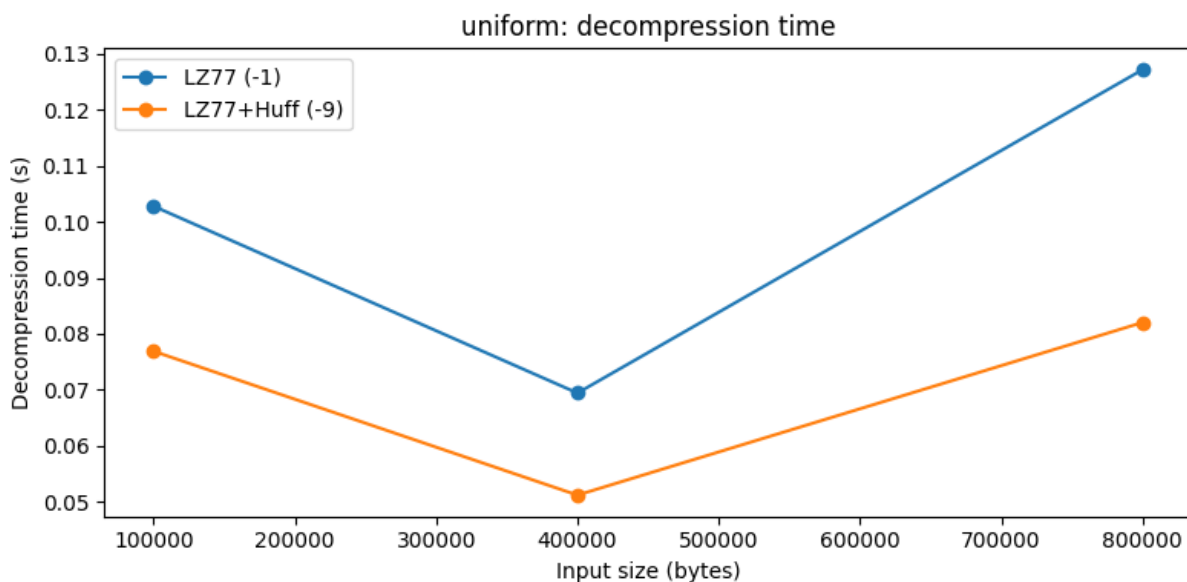
Размер	Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
100 000	-1	1.098	0.103	112 915
	-9	1.141	0.077	107 458
400 000	-1	5.477	0.069	451 484
	-9	5.607	0.051	426 011
800 000	-1	14.121	0.127	902 572
	-9	13.230	0.082	850 910



Коэффициент сжатия (uniform)



Время сжатия (uniform)



Время распаковки (uniform)

Оба режима показывают слабое сжатие ($\text{ratio} \approx 1.06\text{--}1.13$), так как нет повторяющихся последовательностей и частых байтов. Режим -9 немного лучше благодаря Huffman на бит-пакете, но выигрыш небольшой из-за высокой энтропии.

Реальные данные: небольшая текстовая статья (≈ 13 KB)

Ниже результаты на реальном текстовом файле небольшого размера (статья). Текст имеет повторяющиеся слова и фразы, что благоприятно для LZ77.

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	0.110	0.023	7 790
-9	0.114	0.048	8 468

Режим -1 даёт лучшее сжатие ($\text{ratio} \approx 0.59$ против 0.64 у -9). На небольшом файле overhead таблицы Huffman заметен и перевешивает выгоду от дополнительного сжатия.

Реальные данные: большой текстовый отчёт (≈ 606 KB)

Ниже результаты на большом текстовом отчёте. Много повторяющихся структур и фраз — идеально для обоих методов.

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	6.712	0.058	190 113
-9	7.457	0.080	174 359

На большом файле режим -9 выигрывает ($\text{ratio} \approx 0.29$ против 0.31 у -1). Overhead таблицы Huffman уже незначителен, а дополнительное энтропийное сжатие бит-пакета даёт заметный выигрыш.

Реальные данные: PNG-изображение (≈ 1.1 MB)

Ниже результаты на PNG-файле — это уже сжатые данные. Повторений мало, матчи короткие.

Режим	Время сжатия (s)	Время распаковки (s)	Размер сжатого (байт)
-1	37.044	0.230	1 233 595
-9	41.197	0.098	1 225 516

Оба режима слегка увеличивают размер ($\text{ratio} \approx 1.09\text{--}1.10$). Режим -9 немного лучше благодаря Huffman на бит-пакете, но на уже сжатых данных архиватор не может существенно уменьшить объём.

Таким образом архиватор показывает отличные результаты на текстовых данных и синтетике с повторениями. Режим -9 в большинстве случаев превосходит -1 благодаря дополнительному Huffman-кодированию бит-пакета токенов. На небольших файлах overhead таблицы заметен, на больших — выгода от -9 значительна. На уже сжатых данных (PNG) наблюдается небольшое раздутие, но оно минимально в режиме -9.

Недочёты

- Чтение всего файла в память (`read_all`) — не подходит для файлов больше RAM.
- Можно также добавить эвристики для ускорения.

Выводы

Реализованный архиватор сочетает LZ77 и Huffman — классику сжатия без потерь. Отлично работает на текстах с повторениями, плохо на равномерно распределённых случайных и на уже сжатых данных. Основные трудности были связаны с каноническим Huffman и сериализацией.